

Cloud Infrastructure Automation using Terraform and AWS

Services Used:

Terraform
Amazon VPC
Amazon EC2
Amazon RDS
Application Load Balancer
Security Groups
Internet Gateway
Route Tables
Subnets
AWS CLI

Description

This project automates AWS cloud infrastructure deployment using Terraform. The infrastructure includes a custom VPC, public and private subnets, EC2 instance, Application Load Balancer, and Amazon RDS MySQL database. Terraform is used as Infrastructure as Code (IaC) to provision and manage AWS resources automatically.

Definition

Infrastructure as Code (IaC) is the process of managing and provisioning cloud infrastructure using code instead of manual configuration. Terraform automates infrastructure creation, modification, and deletion.

Scope of the Project

- Create custom VPC
- Configure public and private subnets
- Create Internet Gateway
- Configure Route Tables
- Create Security Groups
- Deploy EC2 instance
- Install Nginx automatically
- Deploy RDS MySQL
- Configure Application Load Balancer
- Automate AWS infrastructure using Terraform

AWS Services Used in the Project

1. Terraform

Definition:

Terraform is an Infrastructure as Code tool developed by HashiCorp.

Purpose:

Used to automate AWS infrastructure deployment.

Role:

Creates AWS resources using code.

2. Amazon VPC

Definition:

Amazon VPC is a logically isolated private network in AWS.

Purpose:

Provides secure cloud networking.

Role:

Hosts EC2, subnets, route tables, and RDS.

3. Amazon EC2

Definition:

Amazon EC2 provides virtual cloud servers.

Purpose:

Hosts web applications.

Role:

Runs Nginx web server.

4. Amazon RDS

Definition:

Amazon RDS is a managed relational database service.

Purpose:

Stores application database.

Role:

Hosts MySQL database securely inside private subnet.

5. Application Load Balancer

Definition:

ALB distributes traffic across backend servers.

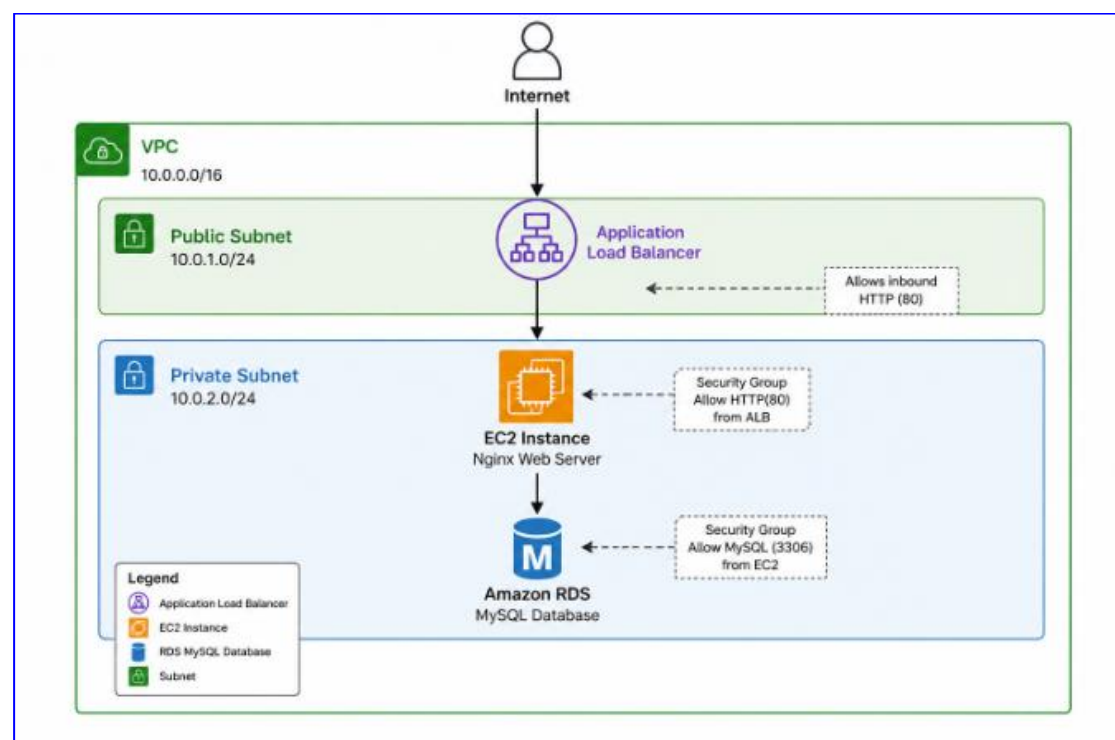
Purpose:

Provides scalability and availability.

Role:

Routes user traffic to EC2 instance.

Architecture Diagram:



Implementation Steps

Step 1: Install Terraform

Commands:

```
sudo yum update -y
sudo yum install -y yum-utils
sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
sudo yum install terraform -y
terraform version
```

Purpose:

Terraform is required to automate AWS infrastructure.

Step 2: Understanding Terraform File Structure

Definition

Terraform project structure organizes infrastructure code into separate files for better readability, management, and reusability.

Files Created

File Name	Purpose
provider.tf	Connects Terraform with AWS
main.tf	Contains main infrastructure resources
variables.tf	Stores reusable variables
terraform.tfstate	Stores infrastructure state
terraform.tfvars	Stores variable values (optional)

Step 3: Configure AWS CLI

Command:

```
aws configure
```

Purpose:

Allows Terraform to authenticate with AWS.

Using access key and secret key.

Step 4: Create VPC using Terraform

Terraform Code:

```
resource "aws_vpc" "main_vpc" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "terraform-main-vpc"
  }
}
```

Purpose:
Creates isolated private network in AWS.

Step 5: Create Public and Private Subnets

Terraform Code:

```
resource "aws_subnet" "public_subnet_1" {
  vpc_id            = aws_vpc.main_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "ap-south-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "public-subnet-1"
  }
}

resource "aws_subnet" "public_subnet_2" {
  vpc_id            = aws_vpc.main_vpc.id
  cidr_block        = "10.0.2.0/24"
  availability_zone  = "ap-south-1b"
  map_public_ip_on_launch = true

  tags = {
    Name = "public-subnet-2"
  }
}

resource "aws_subnet" "private_subnet_1" {
  vpc_id            = aws_vpc.main_vpc.id
  cidr_block        = "10.0.3.0/24"
  availability_zone  = "ap-south-1a"

  tags = {
    Name = "private-subnet-1"
  }
}

resource "aws_subnet" "private_subnet_2" {
  vpc_id            = aws_vpc.main_vpc.id
  cidr_block        = "10.0.4.0/24"
  availability_zone  = "ap-south-1b"

  tags = {
    Name = "private-subnet-2"
  }
}
```

Purpose:
Public subnet hosts internet-facing resources.
Private subnet hosts secure backend resources.

Step 6: Create Internet Gateway

```
resource "aws_internet_gateway" "main_igw">{
  vpc_id = aws_vpc.main_vpc.id

  tags = {
    Name = "main-internet-gateway"
  }
}
```

Purpose:

Allows internet access for public subnet.

Step 7: Configure Route Tables

Terraform Code:

```
resource "aws_route_table" "public_route_table" {
  vpc_id = aws_vpc.main_vpc.id

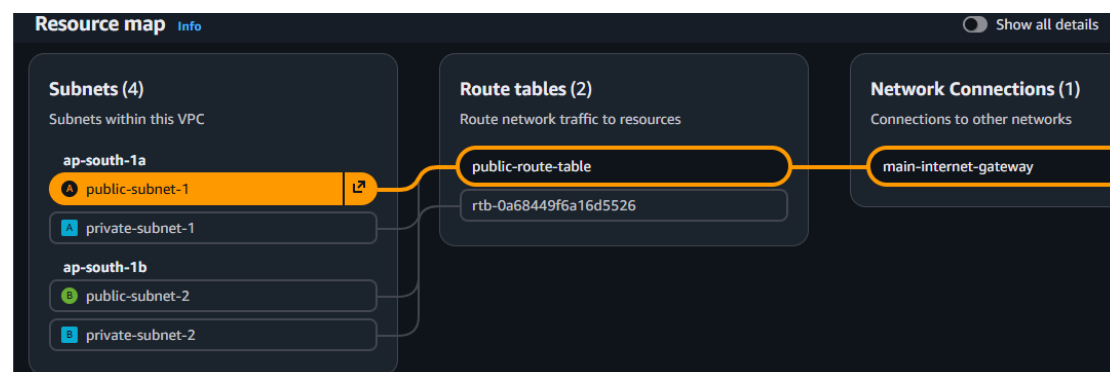
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main_igw.id
  }

  tags = {
    Name = "public-route-table"
  }
}
```

Associate:

```
resource "aws_route_table_association" "public_subnet_1_assoc" {
  subnet_id      = aws_subnet.public_subnet_1.id
  route_table_id = aws_route_table.public_route_table.id
}

resource "aws_route_table_association" "public_subnet_2_assoc" {
  subnet_id      = aws_subnet.public_subnet_2.id
  route_table_id = aws_route_table.public_route_table.id
}
```



Purpose:
Controls network traffic routing.

Step 8: Create Security Groups

Terraform Code:

```
resource "aws_security_group" "ec2_sg" {
  name           = "ec2-security-group"
  description    = "Allow SSH and HTTP"
  vpc_id        = aws_vpc.main_vpc.id

  ingress {
    description = "Allow SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow all outbound"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "ec2-security-group"
  }
}
```

☐ alb-security-group 🔗	sg-09287459db24cf1dc	alb-security-group	vpc-057f4e1f955056fa7 🔗
☐ ec2-security-group	sg-0faca3038db5196	ec2-security-group	vpc-057f4e1f955056fa7 🔗
☐ rds-security-group	sg-0b29f982daffcaae0	rds-security-group	vpc-057f4e1f955056fa7 🔗

Purpose:
Controls inbound and outbound traffic.

Step 9: Launch EC2 Instance

Terraform Code:

```

resource "aws_instance" "web_server" {
  ami           = "ami-0f918f7e67a3323f0"
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.public_subnet_1.id
  vpc_security_group_ids = [aws_security_group.ec2_sg.id]

  user_data = <<-EOF
  #!/bin/bash
  yum update -y
  yum install nginx -y
  systemctl start nginx
  systemctl enable nginx
  echo "<h1>Terraform AWS Project Working</h1>" > /usr/share/nginx/html/index.html
  EOF

  tags = {
    Name = "terraform-web-server"
  }
}

```

Saved filter sets
Choose filter set

Find Instance by attribute or tag (case-sensitive)

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
☐	teraform	i-0fa21bf15c10d2d31	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-1
☒	terraform-web...	i-066861ab4860d83c2	Running	t3.micro	3/3 checks passed	ap-south-1a	-

Purpose:

Deploys web server in AWS.

Also Install Nginx using user_data:

Purpose:

Automatically configures web server during EC2 launch.

Step 11: Create RDS MySQL Database

Terraform Code:

Rds security group:

☐	alb-security-group	sg-09287459db24cf1dc	alb-security-group	vpc-057f4e1f955056fa7
☐	ec2-security-group	sg-0faca3038db5196	ec2-security-group	vpc-057f4e1f955056fa7
☐	rds-security-group	sg-0b29f982daffcaae0	rds-security-group	vpc-057f4e1f955056fa7


```

resource "aws_security_group" "rds_sg" {
  name           = "rds-security-group"
  description    = "Allow MySQL from EC2"
  vpc_id        = aws_vpc.main_vpc.id

  ingress {
    description      = "Allow MySQL from EC2"
    from_port        = 3306
    to_port          = 3306
    protocol          = "tcp"
    security_groups  = [aws_security_group.ec2_sg.id]
  }

  egress {
    from_port        = 0
    to_port          = 0
    protocol          = "-1"
    cidr_blocks      = ["0.0.0.0/0"]
  }

  tags = {
    Name = "rds-security-group"
  }
}

```

Rds subnet group:

```

resource "aws_db_subnet_group" "rds_subnet_group" {
  name = "rds-subnet-group"

  subnet_ids = [
    aws_subnet.private_subnet_1.id,
    aws_subnet.private_subnet_2.id
  ]

  tags = {
    Name = "rds-subnet-group"
  }
}

```

RDS:

```

resource "aws_db_instance" "mysql_db" {
  allocated_storage = 20
  db_name           = "myappdb"
  engine            = "mysql"
  engine_version    = "8.0"
  instance_class    = "db.t3.micro"
  username          = "admin"
  password          = "admin12345"
  db_subnet_group_name = aws_db_subnet_group.rds_subnet_group.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  publicly_accessible = false
  skip_final_snapshot = true

  tags = {
    Name = "terraform-mysql-rds"
  }
}

```

Purpose:

Deploys managed MySQL database securely.

Step 12: Create Application Load Balancer

Terraform Code:

ALB security group:

```

resource "aws_security_group" "alb_sg" {
  name          = "alb-security-group"
  description   = "Allow HTTP traffic"
  vpc_id        = aws_vpc.main_vpc.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "alb-security-group"
  }
}

```

☐ alb-security-group 🔗	sg-09287459db24cf1dc	alb-security-group	vpc-057f4e1f955056fa7 🔗
☐ ec2-security-group	sg-0faca3038d1beb5196	ec2-security-group	vpc-057f4e1f955056fa7 🔗
☐ rds-security-group	sg-0b29f982daffcaae0	rds-security-group	vpc-057f4e1f955056fa7 🔗

Alb Target-group:

```
resource "aws_lb_target_group" "web_tg" {
  name      = "web-target-group"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = aws_vpc.main_vpc.id

  tags = {
    Name = "web-target-group"
  }
}
```

Target groups (1) [Info](#) | [What's new?](#) ⌂ Actions Create

🔍 Filter target groups

☐	Name	ARN	Port	Protocol	Target type
☐	web-target-group	arn:aws:elasticloadbalancin...	80	HTTP	Instance

```
resource "aws_lb_target_group_attachment" "web_tg_attachment" {
  target_group_arn = aws_lb_target_group.web_tg.arn
  target_id        = aws_instance.web_server.id
  port             = 80
}
```

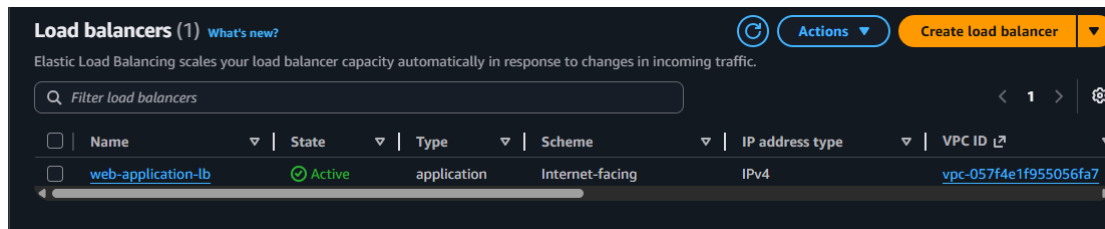
ALB :

```
resource "aws_lb" "web_alb" {
  name                = "web-application-lb"
  internal            = false
  load_balancer_type = "application"

  security_groups = [aws_security_group.alb_sg.id]

  subnets = [
    aws_subnet.public_subnet_1.id,
    aws_subnet.public_subnet_2.id
  ]

  tags = {
    Name = "web-application-lb"
  }
}
```



Purpose:

Distributes traffic to EC2 instance.

Step 13: Create Listener and Target Group

Terraform Code:

Purpose:

Routes traffic from ALB to EC2.

```
resource "aws_lb_listener" "web_listener" {
  load_balancer_arn = aws_lb.web_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.web_tg.arn
  }
}
```

Nginx running :

```
[root@ip-172-31-44-18 ec2-user]# ls
terraform-project
[root@ip-172-31-44-18 ec2-user]# sudo systemctl status nginx
curl http://localhost
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-05-24 15:52:22 UTC; 19min ago
     Main PID: 43438 (nginx)
       Tasks: 3 (limit: 1014)
      Memory: 3.6M
         CPU: 59ms
    CGroup: /system.slice/nginx.service
            └─43438 "nginx: master process /usr/sbin/nginx"
              └─43439 "nginx: worker process"
                └─43440 "nginx: worker process"

May 24 15:52:22 ip-172-31-44-18.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
May 24 15:52:22 ip-172-31-44-18.ap-south-1.compute.internal nginx[43436]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
May 24 15:52:22 ip-172-31-44-18.ap-south-1.compute.internal nginx[43436]: nginx: configuration file /etc/nginx/nginx.conf test is successful
May 24 15:52:22 ip-172-31-44-18.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
<hl>Terraform AWS Project Working</hl>
[root@ip-172-31-44-18 ec2-user]#
```

Troubleshooting

Issue 1: EC2 Free Tier Error

Solution:

Changed instance type from t2.micro to t3.micro.

Conclusion

This project successfully automated AWS infrastructure deployment using Terraform. The project demonstrates Infrastructure as Code, AWS networking, security configuration, EC2 deployment, RDS integration, and Application Load Balancer configuration.